

Navigasyon

Tayvan'da yoğun olarak yüksek dağlar bulunur, 3.000 metrenin üzerinde 250'den fazla zirveye ev sahipliği yapar. A-Ming uzak köyler arasında paket teslimi yapabilmek için Tayvan Merkezi Dağ Bölgesinde robotik zipline teslimat ağı kurmayı planlıyor.

A-Ming'in planlarında, her ağ N nodedan oluşur, 0'dan $N - 1$ 'e numaralandırılır. Bu nodelar arasında, **tam olarak** $K = 6$ tanesi güç istasyonudur ve diğer $N - K$ node köy nodeudur. Nodelar $N - 1$ **yönsüz** kablo ile bağlıdır, 0'dan $N - 2$ 'ye numaralandırılır. Her i için ($0 \leq i \leq N - 2$), kablo i $U[i]$ ve $V[i]$ nodelarını bağlar. Her kablo iki farklı nodeu bağlar, ve her node ikilisi en fazla bir kablo ile bağlıdır. Herhangi bir node ikilisi arasında kabloları kullanarak ulaşımın mümkün olduğu garantidir.

a ve b nodeları arasındaki **distance** (mesafe) $d(a, b)$ ile gösterilir ve aşağıdaki şekilde tanımlanır.

- Eğer $a = b$ ise $d(a, b) = 0$.
- Onun dışında, $d(a, b)$ a 'dan b 'ye hareket etmek için gereken minimum kablo sayısıdır.

Eğer ağdaki her node ya tam olarak 1 yada en az δ kabloya bağlıysa, ağın **branching factor**'ü en az δ deriz. A-Ming B pozitif tam sayısını bilir öyle ki ağın branching factor'ü en az B 'dir.

A-Ming 100 farklı tür kablo hazırlar, $0, 1, \dots, 99$ şeklinde numaralandırılır. Ağdaki her kablo bu türlerden biri ile üretilir.

Robotlar, teslimat görevlerini halledebilmek için zipline kablolarında ilerleyecek. A-Ming robotlara güç istasyonlarına geri dönme ve kendilerini şarj etmeleri için yardımcı olacak bir navigasyon programı yüklemek istiyor. Ne yazık ki, robotların aşırı derecede az hafızası var. Bundan dolayı, navigasyon programını tasarlarken, robotlar sadece, güç istasyonlarının sayısına, garanti edilmiş branching factöre B , ve robotun şuanki konumuna bağlı kablo türlerine bağlı olarak yön bulur.

Bir robot, iki veya daha fazla kabloya bağlı bir köy düğümü u 'da gezinmek istediğinde, öncelikle u 'ya bağlı kabloları rastgele bir sırayla tarar. Programa daha sonra kablo tiplerini sıraya göre içeren bir liste verilir. Her kablo için program, o kablo takip edildiğinde robot ve istasyon arasındaki mesafenin azaldığı güç istasyonlarının sayısını saymalıdır.

Özellikle, diyelim ki $v_1, \dots, v_k$ ($k \geq 2$) u 'ya doğrudan kablo ile bağlı nodelar olsun ve c_i ($1 \leq i \leq k$) u ve v_i nodelarını bağlayan kablonun türü olsun. Programa sonrasında K , B , ve $c_1, c_2, \dots, c_k$

listesi verilecektir. Her i için ($1 \leq i \leq k$), program p güç istasyonlarının sayısını bulmalıdır öyle ki $d(v_i, p) < d(u, p)$.

Sizin göreviniz, kablo türlerini atamak ve navigasyon programını tasarlamak suretiyle bir strateji icat etmektir. Çözümünüzün skoru kullandığınız farklı kablo türüne bağlıdır. (Subtasklar ve Skorlama kısmına detaylar için bakınız).

Kodlama Detayları

İki prosedür kodlamalısınız, bir tane kablo türü atamak için ve bir tane de robotun programı için.

kablo türleri atamak için prosedür şudur:

```
std::vector<int> construct_network(int N, int K, int B,  
                                  std::vector<int> U, std::vector<int> V,  
                                  std::vector<int> P)
```

- N : Nodeların sayısı.
- K : Güç istasyonlarının sayısı.
- B : ağın garanti edilmiş minimum branching factorü.
- U, V : $N - 1$ uzunluğunda diziler, kablo bağlantılarını tanımlar.
- P : $K = 6$ uzunluğunda bir dizi, güç istasyonu nodelarını temsil eder.
- Prosedür her test için **en fazla 10 000 kere** çağırılır.

Bu prosedür $N - 1$ uzunluğunda bir T dizisi dönmelidir:

- $T[i]$ türünde bir kablo $U[i]$ ve $V[i]$ nodelarını bağlamak için kullanılır.
- Her $T[i]$ $0 \leq T[i] < 100$ şartını sağlamalıdır.

robotun programı için kodlamanız gereken prosedür şu şekildedir:

```
std::vector<int> navigate(int K, int B, std::vector<int> C)
```

- K : güç istasyonlarının sayısı.
- B : ağın garanti edilmiş minimum branching factorü.
- C : bir köy nodeuna bağlı tüm kablo türlerinin rastgele bir sırayla listesi.
- C 'nin uzunluğunun en az B olacağı garantidir.
- Bu prosedür her test için en fazla 100 000 kere çağırılır.
- navigate prosedürü orijinal ağ yapısına bağlı kalmayabilir; özellikle, grading system aynı test senaryosu içindeki farklı oluşturulmuş ağlar arasında gezinmek için yapılan tüm çağrıları yeniden sıralayabilir.(The procedure navigate may not rely on the original network structure;

in particular, the grading system may reorder all calls to navigate across the different constructed networks within the same test case.)

Bu prosedür D dizisini döndürmelidir:

- D dizisinin uzunluğu C dizisi ile aynı olmak zorundadır.
- $D[i]$, $C[i]$ 'ye karşılık gelen kablo kullanıldığında robot ile o istasyon arasındaki mesafenin(distance) azaldığı güç istasyonlarının sayısıdır.

Unutmayın ki grading systemde, `construct_network` ve `navigate` prosedürleri **iki ayrı process**de çağırılır.

Kısıtlar

- $K = 6$.
- $K < N \leq 100\,000$.
- Her test için bütün `construct_network` çağrılarındaki N değerleri toplamı 100 000 değerini geçemez.
- $0 \leq U[i] < V[i] < N$ her $0 \leq i \leq N - 2$ için.
- Bir nodedan başka bir nodea kablolar aracılığıyla ulaşmak mümkündür.
- $0 \leq P[0] < P[1] < \dots < P[K - 1] < N$.
- Her test için, `navigate` çağrılarındaki C dizisinin uzunlukları toplamı 200 000 değerini geçemez.

Subtasklar ve Skorlama

Subtask	Skor	Ek Kısıtlar
1	6	$B = 2, N = 7$.
2	14	$B = 2, U[i] = i, V[i] = i + 1$ her i için öyle ki $0 \leq i < N - 1$.
3	20	$B = 5$.
4	20	$B = 4$.
5	40	$B = 2$.

Her test case'de, aşağıdaki şartlardan en azından birisi sağlarsa çözümünüzün skoru o test için 0 olacaktır. (CMS'de `Output isn't correct` olarak raporlanır):

- Herhangi bir `construct_network` çağrısının dönüt değeri geçersiz ise.
- Herhangi bir `navigate` çağrısının dönüt değeri yanlış ise.

Onun dışında, diyelim ki S her `construct_network` çağrısından dönen her T dizisindeki bütün sayılardan büyük olan en küçük sayı olsun. Başka bir deyişle, S en küçük tamsayıdır öyle ki oluşturulan bütün ağlar sadece $0, 1, \dots, S - 1$ türünde kabloları kullanır.

Her subtask için skorunuz S değerine aşağıdaki gibi bağlıdır:

Şart	Subtask 1	Subtask 2	Subtask 3 ve 4	Subtask 5
$100 < S$	0	0	0	0
$16 \leq S \leq 100$	2	2	$12 - \log_2 S$	$24 - 2 \log_2 S$
$10 \leq S \leq 15$	2	4	$16 - 0.5S$	$32 - S$
$7 \leq S \leq 9$	2	6	$21 - S$	$42 - 2S$
$S = 6$	2	10	15	30
$S = 5$	6	14	17.5	40
$S \leq 4$	6	14	20	40

Özellikle, Subtask 1, 2 ve 5'de, eğer $S \leq 5$ ise tam puan alırsınız, ve Subtask 3 ve 4'de, eğer $S \leq 4$ ise tam puan alırsınız.

Örnek

$N = 9$, $K = 6$, ve $B = 2$ olan senaryoyu ele alınız.

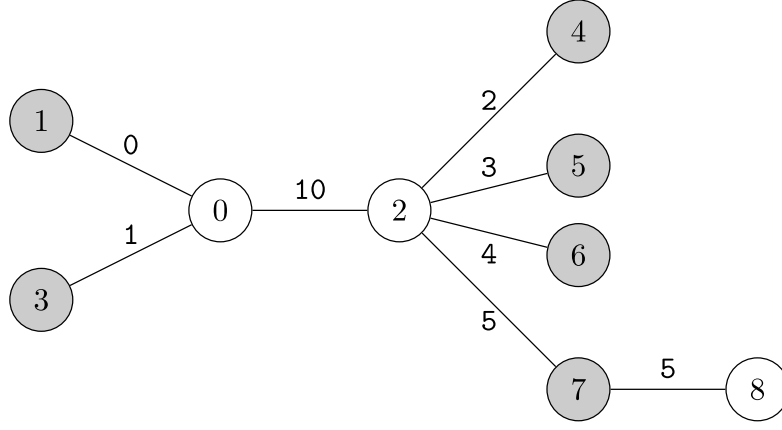
Ağ planı $U = [0, 0, 0, 2, 2, 2, 2, 7]$, $V = [1, 2, 3, 4, 5, 6, 7, 8]$ şeklinde belirtilir. Güç istasyonları $P = [1, 3, 4, 5, 6, 7]$. İlk process için aşağıdaki çağrıyı ele alınız:

```
construct_network(9, 6, 2, [0, 0, 0, 2, 2, 2, 2, 7],
                  [1, 2, 3, 4, 5, 6, 7, 8],
                  [1, 3, 4, 5, 6, 7])
```

Varsayınız ki A-Ming ağı oluşturmak için şu adımları izler:

```
[0, 10, 1, 2, 3, 4, 5, 5]
```

Oluşturulan ağ şu şekilde gösterilir



Bundan sonra, ikinci processde, varsayınız ki robotun 0 nodeunda yol bulması gerekiyor. node 0 node 1, 2 ve 3'e bağlıdır. Eğer robot kablo türlerini bu sırada okursa, aşağıdaki çağrı yapılacaktır:

```
navigate(6, 2, [0, 10, 1])
```

Ağ yapısı ele alındığında:

- İlk güç istasyonu, node 1, kendisine en yakın mesafeye sahiptir.
Özellikle, $0 = d(1, 1) < 1 = d(0, 1) < 2 = d(2, 1) = d(3, 1)$.
- İkinci güç istasyonu, node 3, kendisine en yakın mesafeye sahiptir.
Özellikle, $0 = d(3, 3) < 1 = d(0, 3) < 2 = d(1, 3) = d(2, 3)$.
- Diğer güç istasyonları, node 4, 5, 6, ve 7, node 2'ye daha kısa mesafeye sahiptir.
Özellikle, $1 = d(2, u) < 2 = d(1, u) < 3 = d(1, u) = d(3, u)$ when $u = 4, 5, 6$, veya 7.

(0, 1), (0, 2), ve (0, 3) kabloları takip edildiğinde o istasyona mesafenin azaldığı güç istasyonu sayısı, kablolar için sırasıyla 1, 4 ve 1 olur. Bundan dolayı, prosedür şunu döndürmelidir:

```
[1, 4, 1]
```

Unutmayın ki `navigate` C 'nin farklı bir permutasyonu ile de çağrılabilirdi. Örneğin, `navigate(6, 2, [1, 0, 10])` 0 nodeu için mümkün bir çağrı örneğidir. Prosedür bu durumda `[1, 1, 4]` döndürmelidir.

Varsayalım ki robot node 2'den yön bulmak istedi. Aşağıdaki çağrı yapılır:

```
navigate(6, 2, [10, 2, 3, 4, 5])
```

Prosedür şunu döndürmelidir:

```
[2, 1, 1, 1, 1]
```

Bu aşda, prosedür `navigate` diğer nodelar için asla çağrılmamalıdır çünkü:

- node 1, 3, 4, 5, 6 ve 7 güç istasyonlarıdır ve
- node 8 sadece bir kabloya bağlıdır.

Bu test durumunda, kullanılan kablo türleri 0, 1, 2, 3, 4, 5, ve 10. Bu nedenle, $S = 11$ skoru belirlemek için kullanılacaktır..

Problem için verilen attachment paketi iki farklı girdi ve çıktı içerir ve örnek grader'ı da içerir.

Örnek Grader

Örnek grader `construct_network` ve `navigate`'i aynı process içinde çağırır. Ek olarak, örnek grader `navigate`'i çağırıldığında, kablo türleri inputtaki ilgili kabloların sırasına göre listelenir. İki davranış da gerçek (actual) grader'dan farklıdır.

Girdi formatı:

```
T
(scenario 0)
(scenario 1)
...
(scenario T-1)
```

Her senaryo şu formattadır:

```
N
B
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
K
P[0] P[1] ... P[K-1]
Q
X[0] X[1] ... X[Q-1]
```

Her senaryoda, örnek grader `navigate`'i Q kere çağırır, i -inci çağrıda node $X[i - 1]$ bilgisi ile. Unutmayın ki örnek grader K 'yı 6'dan başka bir sayıya eşitlemeye de elverişlidir.

Çıktı formatı:

Eğer herhangi bir `construct_network` veya `navigate`'dan gelen dönüt dizisi geçersiz ise, örnek grader çöker ve sebebini yazdırır. Onun dışında, örnek grader satırda `navigate` çağrısından dönen her D dizisini çıktı verir.

Her senaryo işlendikten sonra, örnek grader standart errora bir satır yazar:

```
S = S'
```

S' testteki `construct_network` çağrısından dönen her dizideki bütün sayılardan büyük olan en küçük sayıdır.